

+++ date = '2026-03-13T20:47:47-07:00' draft = false title = 'Practica3' +++

Reporte: Instalación del entorno y aplicación TODO en Haskell

1. Instalación del entorno

Para trabajar con Haskell se utilizó la herramienta **Stack**, la cual permite gestionar proyectos, dependencias y la versión del compilador.

Pasos realizados

1. Instalación de Stack desde la [página oficial de Haskell](#).
2. Verificación de la instalación:

```
stack --version
```

3. Creación del proyecto:

```
stack new mi-proyecto
```

4. Acceso al proyecto:

```
cd mi-proyecto
```

5. Compilación del proyecto:

```
stack build
```

6. Ejecución del programa:

```
stack run
```

Durante este proceso, Stack descargó automáticamente el compilador **GHC** y configuró el entorno de desarrollo.

Imágenes — Instalación del entorno

```

You can provide them in Stack's global configuration file (C:\Users\Sebas\AppData\Roaming\stack\config.yaml) like this:

templates:
  params:
    author-email: value
    author-name: value
    category: value
    copyright: value
    github-username: value

Or you can pass each one on the command line as parameters like this:

stack new mi-proyecto new-template -p "author-email:value" -p "author-name:value" -p "category:value" -p "copyright:value" -p
"github-username:value"

Looking for Cabal or package.yaml files to use to initialise Stack's project-level YAML configuration file.
Using the Cabal packages:
* mi-proyecto\
Selecting the best among 14 snapshots...

Note: Matches https://raw.githubusercontent.com/commercialhaskell/stackage-snapshots/master/lts/24/38.yaml

Selected the snapshot https://raw.githubusercontent.com/commercialhaskell/stackage-snapshots/master/lts/24/38.yaml.
Initialising Stack's project-level configuration file using snapshot
https://raw.githubusercontent.com/commercialhaskell/stackage-snapshots/master/lts/24/38.yaml.
Considered 1 user package.
Writing configuration to mi-proyecto\stack.yaml.
Stack's project-level configuration file has been initialised.
Sebas> cd mi-proyecto
mi-proyecto> stack build
mi-proyecto> configure (lib + exe)
Configuring mi-proyecto-0.1.0.0...
mi-proyecto> build (lib + exe) with ghc-9.10.3
Preprocessing library for mi-proyecto-0.1.0.0...
Building library for mi-proyecto-0.1.0.0...
[1 of 2] Compiling Lib
[2 of 2] Compiling Paths_mi_proyecto
Preprocessing executable 'mi-proyecto-exe' for mi-proyecto-0.1.0.0...
Building executable 'mi-proyecto-exe' for mi-proyecto-0.1.0.0...
[1 of 2] Compiling Main
[2 of 2] Compiling Paths_mi_proyecto
[3 of 3] Linking .stack-work\dist\1a191874\build\mi-proyecto-exe\mi-proyecto-exe.exe
mi-proyecto> copy/register
Installing library in C:\Users\Sebas\mi-proyecto\.stack-work\install\e8954d95\lib\x86_64-windows-ghc-9.10.3-b42a\mi-proyecto-0.1.0.0-CeuDo10P2JR1FV0ExHRG0a
Installing executable mi-proyecto-exe in C:\Users\Sebas\mi-proyecto\.stack-work\install\e8954d95\bin
Registering library for mi-proyecto-0.1.0.0...
mi-proyecto> stack exec mi-proyecto-exe
someFunc
Sebas> mi-proyecto CONFIG NOT FOUND
in push at 18:38:50

```

2. Descripción de la aplicación TODO

La aplicación implementada es un sistema básico de **lista de tareas (TODO list)** escrito en Haskell, utilizando el paradigma funcional.

Estructura del proyecto

Archivo	Descripción
app/Main.hs	Punto de entrada del programa
src/Lib.hs	Contiene la lógica principal

Imágenes — Estructura del proyecto

![alt text](image-2.png)

3. Funcionamiento de la aplicación

La aplicación trabaja con una estructura simple:

```
type Todo = [String]
```

Esto representa una **lista de tareas**.

Funcionalidades implementadas

3.1 Agregar tarea

Permite insertar una nueva tarea en la lista:

```
addTask :: String -> Todo -> Todo
addTask task list = task : list
```

3.2 Mostrar tareas

Muestra todas las tareas almacenadas:

```
showTasks :: Todo -> String
showTasks [] = "No tasks"
showTasks (x:xs) = x ++ "\n" ++ showTasks xs
```

3.3 Eliminar tarea

Permite eliminar una tarea específica:

```
removeTask :: String -> Todo -> Todo
removeTask _ [] = []
removeTask task (x:xs)
  | task == x = xs
  | otherwise = x : removeTask task xs
```

4. Ejecución del programa

En el archivo `Main.hs`, se realizan pruebas de la aplicación:

```
main :: IO ()
main = do
  let tasks = []
  let tasks1 = addTask "Estudiar Haskell" tasks
  let tasks2 = addTask "Hacer tarea" tasks1

  putStrLn (showTasks tasks2)
```

Imágenes — Ejecución

```
mi-proyecto> stack build
mi-proyecto> build (lib + exe) with ghc-9.10.3
Preprocessing library for mi-proyecto-0.1.0.0...
Building library for mi-proyecto-0.1.0.0...
[1 of 2] Compiling Lib [Source file changed]
src\Lib.hs:9:1: warning: [GHC-40910] [-Wunused-top-binds]
    Defined but not used: 'someFunc'
9 | someFunc = putStrLn "someFunc"
  | ^^^^^^^^

Preprocessing executable 'mi-proyecto-exe' for mi-proyecto-0.1.0.0...
Building executable 'mi-proyecto-exe' for mi-proyecto-0.1.0.0...
[1 of 2] Compiling Main [Source file changed]
app\Main.hs:1:1: warning: [GHC-85401] [-Wmissing-export-lists]
    The export item 'module Main' is missing an export list
1 | module Main where
  | ^

[3 of 3] Linking .stack-work\dist\1a191874\build\mi-proyecto-exe\mi-proyecto-exe.exe [Objects changed]
mi-proyecto> copy/register
Installing library in C:\Users\Sebas\mi-proyecto\.stack-work\install\e8954d95\lib\x86_64-windows-ghc-9.10.3-b42a\mi-proyecto-0.1.0.0-CeuDo10P2JR1FV0ExHRG0a
Installing executable mi-proyecto-exe in C:\Users\Sebas\mi-proyecto\.stack-work\install\e8954d95\bin
Registering library for mi-proyecto-0.1.0.0...
mi-proyecto> stack run
Hacer tarea
Estudiar Haskell
No tasks
Sebas ➤ mi-proyecto CONFIG NOT FOUND | in pwsh at 19:02:07
```

5. Conclusión

Haskell permite implementar aplicaciones mediante **programación funcional**, donde los datos no se modifican directamente, sino que se generan nuevas versiones de las estructuras.

La aplicación TODO demuestra el uso de:

- **Listas** como estructura de datos principal
- **Recursión** para recorrer y manipular las listas
- **Composición de funciones** para construir el flujo del programa
- **Stack** como gestor de proyectos y compilación